

StudyMate API

Résumé complet du projet — MLOps Junior

Pipeline global

Étape	Outil	Résultat
1. Dev local	FastAPI + ChromaDB + Groq	API RAG fonctionnelle
2. Container	Docker + Docker Compose	App isolée, reproductible
3. CI/CD	GitHub Actions	Tests + image auto sur push
4. Registry	Docker Hub	Image disponible partout
5. Orchestration	Kubernetes (Minikube)	2 replicas, auto-restart
6. IaC	Terraform	Infra créée en 1 commande

Étape 1 — FastAPI

Construction de l'API REST avec 4 endpoints et un pipeline RAG complet.

Architecture RAG

User → FastAPI → ChromaDB (retrieval) → Groq LLM (generation) → Response

Endpoints

Méthode	Route	Description
GET	/	Health check
POST	/upload	Upload PDF → chunking → embeddings → ChromaDB
POST	/ask	Question → retrieval → LLM → réponse + logs
GET	/stats	Statistiques LLMops (latence, tokens, requêtes)

Commande clé

```
uvicorn app.main:app --reload
```

Tester sur : <http://localhost:8000/docs> (Swagger UI intégré automatiquement par FastAPI)

Fichiers du projet

Fichier	Rôle
app/main.py	Endpoints FastAPI

app/rag.py	ChromaDB + chunking + retrieval
app/llm.py	Client Groq (génération)
app/monitoring.py	Logs JSONL + stats LLMops
tests/test_api.py	Tests pytest (5 tests)

Étape 2 — Docker + Docker Compose

Containeriser l'app pour qu'elle tourne identiquement sur n'importe quelle machine.

Dockerfile — recette de l'image

```
FROM python:3.12-slim
RUN apt-get install build-essential gcc g++ cmake git
COPY requirements.txt . && pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

docker-compose.yml — lancer tout ensemble

Lance 2 services en même temps et les fait communiquer :

- app → ton API FastAPI (port 8000)
- chromadb → base vectorielle (port 8001)

```
docker compose up --build
```

Pourquoi Docker ?

Sans Docker	Avec Docker
Marche sur ton PC seulement	Marche partout (Mac, Linux, cloud)
"Fonctionne chez moi..."	Identique partout
Dépendances manuelles	Tout emballé dans l'image

Étape 3 — CI/CD avec GitHub Actions

Automatiser les tests, le build et le push à chaque git push sur main.

Fichier : .github/workflows/ci.yml

2 jobs s'enchaînent automatiquement :

Job	Ce qu'il fait	Si échoue
test	Install deps → pytest tests/ -v	Pipeline s'arrête, rien n'est déployé

build	docker build → docker push Docker Hub	Ne se lance que si test réussit
-------	---------------------------------------	---------------------------------

Secrets GitHub requis

Settings → Secrets and variables → Actions → New repository secret :

- GROQ_API_KEY → ta clé Groq
- DOCKER_USERNAME → mahmoudazizammar
- DOCKER_PASSWORD → ton Access Token Docker Hub

Flux complet

git push → GitHub Actions → pytest → docker build → docker push → image disponible sur Docker Hub

Étape 4 — Kubernetes avec Minikube

Simuler un déploiement production en local. Kubernetes gère les containers automatiquement.

Commandes dans l'ordre

```
minikube start --driver=docker
kubectl create secret generic studymate-secrets --from-literal=GROQ_API_KEY=ta_clé
kubectl apply -f k8s/
minikube service studymate-service
```

Les 3 fichiers k8s/

Fichier	Rôle	Points clés
deployment.yaml	Lance les pods	2 replicas, image Docker Hub, clé Groq
service.yaml	Expose l'app	LoadBalancer, génère une URL d'accès
configmap.yaml	Configuration	Variables non-secrètes partagées

Pourquoi Kubernetes vs Docker Compose ?

Docker Compose	Kubernetes
1 seule machine	Plusieurs serveurs
Pas de restart auto	Restart automatique si crash
Mise à jour = interruption	Rolling update sans interruption
Dev / test	Production

Étape 5 — Terraform (Infrastructure as Code)

Décrire toute l'infrastructure dans un fichier et la créer en une commande.

Commandes

```
terraform init
terraform plan
terraform apply
```

Ce que fait terraform/main.tf

- Télécharge l'image Docker Hub : mahmoudazizammar/studymate-api:latest
- Crée le container avec le bon port (8080→8000)
- Passe la clé GROQ_API_KEY en variable sécurisée

Sans Terraform vs Avec Terraform

Sans Terraform	Avec Terraform
kubectl apply (manuel)	terraform apply (automatique)
kubectl create secret (manuel)	variable déclarée dans le fichier
docker run (manuel)	resource docker_container {}
Commandes à retaper à chaque fois	Un fichier versionné sur GitHub

Stack complète — Résumé

Outil	Couche	Commande clé	Pour le CV
FastAPI	API	uvicorn app.main:app	API LLM / RAG
ChromaDB	Vector DB	PersistentClient()	Base vectorielle
Groq	LLM	llm.py	LLM rapide et gratuit
Docker	Container	docker compose up	Containerisation
GitHub Actions	CI/CD	git push	Automatisation
Kubernetes	Orchestration	kubectl apply -f k8s/	Déploiement prod
Terraform	IaC	terraform apply	Infra as Code
monitoring.py	LLMOps	/stats endpoint	Observabilité LLM